

Dependency-Aware Code Smell Refactoring with an Adaptive Coding-Agent Harness

Havriil Pietukhin

*Department of Applied Informatics
Faculty of Mathematics, Physics and Informatics,
Comenius University in Bratislava
0009-0000-7504-9720
pietukhin1@uniba.sk*

Ivan Polášek

*Gratex International,
Department of Applied Informatics
Faculty of Mathematics, Physics and Informatics,
Comenius University in Bratislava
0000-0001-6004-701X
ipo@gratex.com*

Abstract—Refactoring one code smell can remove or introduce others, so operation order matters. We extend a coding-agent harness with dependency-aware planning, Java editing, and an automatic verification loop. Each Java edit triggers the project tests and the Organic smell detector. The harness returns both results to the coding agent before it continues. A smell may receive five attempts. The harness changes from a low-cost model to a stronger model after two failed attempts. We evaluate four ordering methods on executable Java episodes reconstructed from the Neo4j representation of the Sousa composite-refactoring dataset. Across 20 completed runs, the harness removed all 350 initial smell detections and all 220 detections introduced during editing. Compilation failed during 171 of 704 intermediate verifications, and 14 runs had at least one failure. The harness recovered from these intermediate failures, but its edits were not error-free. For the episodes evaluated with Best-First Search, it introduced fewer smells than detector order and required fewer edit tasks than topological ordering.

Index Terms—code smells, automated refactoring, coding agents, search-based software engineering, static analysis

I. INTRODUCTION

Code smells [1] are symptoms of poor object-oriented design. In long-lived projects, they accumulate, make code harder to read, and raise maintenance cost. More than 59% of smell-bearing classes contain multiple smells [2]. Combined smells hurt comprehension much more than isolated smells [3]. Recent large language model (LLM)-based tools automate individual refactorings [4], [5], but they usually treat each smell independently.

Cedrim et al. [6] found that 57% of refactorings in a longitudinal study of 23 Java projects did not reduce the smell count; some increased it. Operation order matters because smells interact. Refactoring one smell can remove or introduce others, depending on the current class structure. Markovič [7] formalised these positive and negative dependency rules. We use that catalogue for dependency candidates, restricted to the smell types and refactoring operations listed in our tables.

We study four questions.

(RQ1) Planner choice. How does the planning algorithm affect whether the harness removes all detected smells while the repository tests pass?

(RQ2) Dependency rules. How do the positive and negative dependency rules affect the same outcome?

(RQ3) Predicted and observed dependencies. Do different planners produce the smell removals and introductions predicted by the rule catalogue?

(RQ4) Structural quality. How do the resulting code states differ in CBO, LCOM, WMC, and LOC?

Our implementation extends a coding-agent harness. It builds a directed dependency graph, orders detected smells, asks the coding model to edit one smell at a time, and checks each edit before continuing. Its control loop returns test and detector results to the coding model before selecting the next smell. The ordering algorithms use the same dependency graph.

II. RELATED WORK

A. Code smell detection

Lanza and Marinescu [8] systematised threshold-based detectors built on the metrics of Chidamber and Kemerer [9]. Fontana et al. [10] reported 96% SVM accuracy across 74 Java projects. Yamashita and Moonen [11] and Palomba et al. [2] studied co-located smells. Palomba et al. found multiple smell types in 59% of smell-bearing classes. Abbes et al. [3] measured a 124% reduction in comprehension for combined smells.

B. Code refactoring

Cedrim et al. [6] annotated 16,566 refactorings in 23 projects as smell-reducing, smell-introducing, or neutral. Of these operations, 57% did not reduce the smell count, and some increased it. Execution order can decide whether automated refactoring improves or worsens a codebase. Markovič [7] derived a catalogue of positive dependencies (refactoring smell *A* may resolve smell *B* as a side effect) and negative dependencies (refactoring *A* may introduce *B*),

grounded in Fowler’s refactoring catalogue [1]. Negative edges depend on state. A negative dependency applies only when the class still has the required structure. Removing prerequisite smells first can suppress the edge. The catalogue gives candidate effects rather than measured probabilities. We use it to order operations, then redetect smells after each actual edit.

C. Large language models for code

iSMELL [4] combines Mixture-of-Experts with static analysis and reports $F1 = 75.17\%$ on seven smell types. SmellDetector [5] (IJCNN 2025) fine-tunes LLMs for multi-label smell detection across Java projects. Both target individual smell instances; neither models interactions between sequential refactoring operations or uses dependency-aware planning.

The coding harness also affects cost and task completion. Gaba et al. [12] ran the same model and reasoning effort through several harnesses on internal Databricks tasks. Pi sent about one third as much context per turn. Some configurations differed in cost by more than a factor of two at equal measured quality. Their tasks differ from ours, so we use these results only to justify the harness choice.

III. OUR APPROACH

A. Method overview

The implementation uses Pi, a terminal coding-agent harness with a TypeScript extension interface and model switching [13]. In the Databricks benchmark, Pi matched measured quality while reducing context use and, in some configurations, cost [12]. Pi separates model access from agent execution and loads workflow changes through extensions [13]. Our extension prepares a dataset case, presents one smell, monitors Java changes, runs verification, and returns the result to the coding model. Fig. 1 shows this loop.

The dependency rules predict which smells one refactoring may remove or introduce. DAPS, topological ordering, and BeFS use those predictions to select the next smell. The harness requests the edit, checks it, reports failures, and retries when needed. Organic then redetects the smells. Its output, not the predicted transition, defines the next state.

The harness checks out the recorded start commit in an isolated worktree. The Organic detector [14] scans the selected classes and methods. A task includes the smell type, severity, owner, source range, detector evidence, and refactoring advice. The model receives only the current smell turn. The harness uses the tests already present in each repository and does not generate new tests.

B. Smell dependency model

We represent smell dependencies as a directed graph $G = (V, E)$, where each node $s_i \in V$ is a detected smell instance. The graph has two edge types:

- *positive* $s_i \xrightarrow{+} s_j$: refactoring s_i may resolve s_j as a side effect;
- *negative* $s_i \xrightarrow{-} s_j$: refactoring s_i may introduce s_j .

We add an edge when the smell-type pair matches a rule from the catalogue [7] and both smells are in the same class

TABLE 1. Dependency rules [7].

Smell	Positive (resolves)	Negative (introduces)
Long Method / Complex	Feature Code, Long List	Envy, Dup. Param. List
Long Param. List	Data Clumps	Data Class
God Class	Feature Envy, Data Clumps	Long Method, Data Class, Inapp. Intimacy
Large Class	Divergent Change	Large Class
Dup. Conditions		

context. A positive edge predicts that one refactoring may remove another smell. A negative edge predicts that it may introduce one. These predictions rank operations. After each Java edit, Organic replaces the predicted state with detector output. We treat negative edges as more reliable because a positive side effect may not occur in a given class. Positive edges use co-located source instances. Negative edges use the abstract catalogue rules. The model does not include transitive positive dependencies. Table 1 lists the implemented rules.

C. Refactoring-order algorithms

The ordering algorithms give the harness one detected smell at a time. Dependency-Aware Priority Selection (DAPS) assigns each smell s_i a priority score $P(s_i)$ and chooses the highest-scoring smell. The general form is:

$$P_i = f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos_out}(s_i) - w_{\text{neg}} \cdot \sum \text{neg_out}(s_i)), \quad (1)$$

where $f_i = \text{freq}(s_i)$ counts occurrences of smell s_i . $w_{\text{sev}}=0.33$ normalises the severity scale. $w_{\text{neg}}=0.5$ discounts predicted introductions because they depend on class structure. pos_out and neg_out count outgoing positive and negative edges.

Positive edges require co-located smell instances in the source file. Negative edges follow Markovič’s abstract catalogue because their structural risk does not depend on observed co-location. The concrete priority formula is:

$$P_i^{\text{conc}} = f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos_out}^{\text{conc}}(s_i) - w_{\text{neg}} \cdot \sum \text{neg_out}^{\text{abs}}(s_i)). \quad (2)$$

Future calibration could weight each positive edge by its observed probability using data such as that of Cedrim et al. [6]. DAPS selects $\arg \max P_i$ at each step. This local choice may produce a poor global order when the selected smell has active negative edges.

1) *Topological ordering*: The topological treatment follows the scored ordering proposed by Orlova and Poláček [15]. It converts dependency edges into refactoring-preference constraints, condenses strongly connected components, and scores candidate topological orders by positive-edge distance and vertex weight. Our implementation enumerates at most 4,096 orders and selects the order with the largest sum of normalised distance and vertex scores. It sends the first smell in that order to the coding model.

2) *Dependency-Aware Priority Selection*: Algorithm 1 formalises DAPS.

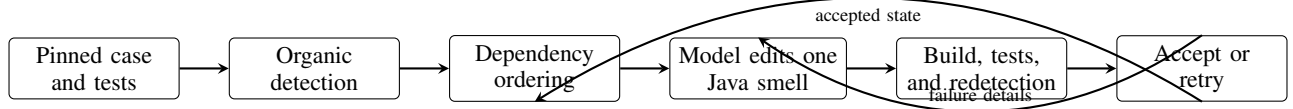


Fig. 1. Modified coding-agent control loop. The detector and project tests produce each accepted state; predicted planner transitions do not.

Algorithm 1 Dependency-Aware Priority Selection (DAPS).

Require: Initial state S_0 , dependency rules R

Ensure: Ordered refactoring plan π

```

1:  $S \leftarrow S_0$ ;  $\pi \leftarrow []$ 
2: while  $S \neq \emptyset$  do
3:   for each  $s_i \in S$  do
4:      $P(s_i) \leftarrow f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos\_out}^{\text{conc}}(s_i)$ 
5:        $- w_{\text{neg}} \cdot \sum \text{neg\_out}^{\text{abs}}(s_i))$ 
6:   end for
7:    $s^* \leftarrow \arg \max_{s_i \in S} P(s_i)$ 
8:    $\pi \leftarrow \pi ++ [r(s^*)]$ 
9:    $S \leftarrow (S \setminus \text{resolved}(s^*, S)) \cup \text{created}(s^*, S)$ 
10: end while
11: return  $\pi$ 

```

3) *Best-First Search (BeFS) algorithm:* Algorithm 2 treats refactoring as state-space search:

- **State** S : current set of active smell instances;
- **Action** $r(s_i)$: refactor smell $s_i \in S$;
- **Transition**: $S' = (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$, where $\text{resolved}(s_i, S)$ includes s_i itself plus any smells in S that are positively resolved as a side effect;
- **Heuristic**: $h(S) = \sum_{s \in S} \text{sev}(s)$;
- **Objective**: minimise $h(S)$; the search seeks the lowest achievable total severity, which need not reach zero.

BeFS orders states by $h(S)$, not by the per-smell priority $P(s_i)$. Only DAPS uses this score to choose the next smell (Algorithm 1). The implementation stops after at most $N_{\text{max}}=10,000$ state expansions. The CLOSED set prevents revisiting an expanded state.

Fig. 2 compares Algorithms 1 and 2 on a four-smell example. God Class (GC), Long Method (LM), Feature Envy (FE), and Data Clumps (DC), where subscripts H and M denote HIGH and MEDIUM severity. $S_0 = \{GC_H, LM_H, FE_M, DC_M\}$, $h=10$. Using $w_{\text{sev}}=0.33$, $w_{\text{neg}}=0.5$, with $f_{GC}=2$ (two God Class detections) and all other $f_i=1$: $P(GC)=2 \cdot (0.33 \cdot 3 + 1 - 0.5 \cdot 2)=1.98$; $P(LM)=1 \cdot (0.33 \cdot 3 + 1 - 0.5 \cdot 1)=1.49$; $P(FE)=1 \cdot (0.33 \cdot 2 + 0 - 0)=0.66$; $P(DC)=1 \cdot (0.33 \cdot 2 + 0 - 0)=0.66$.

DAPS trace. DAPS selects $\arg \max P$, which is GC ($P=1.98$). Extract Class fires two negative dependency edges, introducing LM' and Inappropriate Intimacy (II). $S_1^G = \{LM, LM', II\}$, $h=8$; the ordering must then handle three smells over at least four remaining steps.

Algorithm 2 BeFS algorithm.

Require: Initial state S_0 , dependency rules R , expansion limit N_{max}

Ensure: Lowest- h plan found within N_{max} expansions

```

1: OPEN  $\leftarrow \{(h(S_0), S_0, [])\}$ 
2: CLOSED  $\leftarrow \emptyset$ ;  $n \leftarrow 0$ ;  $\pi^* \leftarrow []$ ;  $h^* \leftarrow h(S_0)$ 
3: while OPEN  $\neq \emptyset \wedge n < N_{\text{max}}$  do
4:    $(h, S, \pi) \leftarrow \text{extract\_min}(\text{OPEN})$ 
5:   if  $h < h^*$  then
6:      $h^* \leftarrow h$ ;  $\pi^* \leftarrow \pi$ 
7:   end if
8:   if  $S \in \text{CLOSED}$  then
9:     continue
10:  end if
11:  CLOSED  $\leftarrow \text{CLOSED} \cup \{S\}$ 
12:   $n \leftarrow n + 1$ 
13:  for each  $s_i \in S$  do
14:     $S' \leftarrow (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$ 
15:    if  $S' \notin \text{CLOSED}$  then
16:      push(OPEN,  $(h(S'), S', \pi ++ [r(s_i)])$ )
17:    end if
18:  end for
19: end while
20: return  $\pi^*$ 

```

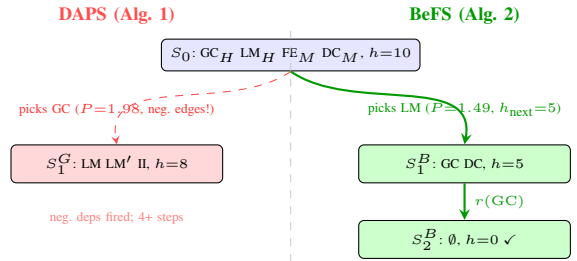


Fig. 2. DAPS vs. BeFS ordering trace.

BeFS trace. BeFS evaluates all successors of S_0 . Expanding $r(LM)$ gives $\text{resolved}=\{LM, FE\}$ (positive edge), $\text{created}=\emptyset$; $S_1^B=\{GC, DC\}$, $h=5$. Expanding $r(GC)$ from S_1^B gives $\text{resolved}=\{GC, DC\}$, $\text{created}=\emptyset$ (LM already removed, so negative edges suppressed); $S_2^B=\emptyset$, $h=0$. The path takes 2 steps and introduces no smells.

D. LLM-based refactoring agent

The harness starts each smell with a lower-cost model. After a failed attempt, it returns the current diff, test status,

current-smell status, and remaining detections to the coding agent. After two failures, it selects a higher-capability model. Each smell receives at most five attempts and 30 model turns per attempt. An HTTP 429 response delays and repeats the same attempt without consuming the attempt budget. After the fifth failure, the harness skips the smell. It marks the case as exhausted after skipping every remaining detection.

Every Java change triggers verification, whether it came from an edit tool, write tool, or shell command. The harness runs the project tests and Organic. For Maven-only inputs, it derives a Gradle build and verifies through Gradle. It blocks direct edits to build files and whole-worktree rollback commands. The harness moves to the next smell only when

$$\begin{aligned} \text{accept}(e, s) = & \text{javaEdit}(e) \wedge \text{latestVerified}(e) \\ & \wedge \text{testsAcceptable}(e) \wedge s \notin D(e), \end{aligned} \quad (3)$$

where e is the latest edit, s is the assigned smell, and $D(e)$ is the detector output after that edit. The gate is binary and has no numeric threshold. It rejects an edit if the assigned smell remains, even when the total smell count falls.

Actual edits can differ from predicted transitions. Redetection replaces the prediction before the next ordering decision.

IV. EXPERIMENT

Dataset. The evaluation uses the Composite Refactorings 2020 dataset of Sousa et al. [16]. It links refactorings, commits, changed or produced program elements, and smell states from 48 Java projects. We use its range-based construction. Refactorings by the same developer form a composite when their affected element sets overlap across consecutive developer commits; this overlap can extend the composite transitively. Here, range means a commit span connected through overlapping program elements, not a source-line range. From these composites, we select a bounded class or method scope and check out its pre-refactoring commit. The ten buildable episodes are range-seeded, selected-scope cases, not a random sample or an exact replay of every full composite.

Detector-order baseline. The harness processes smells in Organic’s native detector order without dependency-based reordering.

Planned runs. The same harness instead uses Dependency-Aware Priority Selection (DAPS), topological ordering, or BeFS. Each condition uses the same detector, prompt structure, weak model, escalation rule, attempt limit, and verification gate. We count original and introduced smells, model turns, edit tasks, verification outcomes, token use, elapsed time, and final CK metrics. The CK static-analysis tool [17] supplies CBO for coupling, LCOM for lack of cohesion, WMC for class complexity, and LOC for size. The experiment has one run per combination of ordering and episode, so it does not estimate variation from model sampling.

Comparing detector-order and planned runs measures the dependency rules and search procedure together. The harness remains fixed. Verification logs describe its recovery from failed edits, but the experiment has no second harness for a causal comparison.

Order	Runs	Original	Introduced	Failed verify
BeFS	2	32/32	30/30	12/60
DAPS	6	106/106	67/67	46/202
Detector	6	106/106	65/65	67/224
Topo	6	106/106	58/58	46/218
Total	20	350/350	220/220	171/704

TABLE 2. Smell removal and failed intermediate verifications in completed runs. Fractions show fixed/observed smells and failed/total verifications.

Order	Runs	Tasks	Turns	Tokens	Minutes
BeFS	2	39	366	1,571,971	67.7
DAPS	6	95	1,035	6,263,115	174.3
Detector	6	92	1,090	6,661,949	144.8
Topo	6	102	1,103	6,314,177	146.7
Total	20	328	3,594	20,811,212	533.5

TABLE 3. Observed work and token use for completed runs. BeFS totals cover two episodes and are not directly comparable with the six-episode totals.

A completed run has no remaining original or introduced smell and has final CK output. It ends after the harness removes every detected smell. The definition has no tunable reduction threshold. It measures smell removal under the repository tests, not semantic equivalence.

A. Completion and recovery

The 4×10 matrix produced 16 successful, eight failed, and 16 aborted runs. Case-level runs added four completed observations, giving 20 completed runs across six episodes from four projects. BeFS contributes two observations; each other ordering contributes six. The analysis includes failed and aborted outcomes as well as completed runs.

Every completed run removed all detected smells, including those introduced during editing. Completion often required more than one edit. Compilation failed during 171 of 704 verification attempts (24.3%). Fourteen of 20 runs had at least one failure. Eight runs needed a second or third attempt for at least one smell, and three reached the third attempt. No completed run recorded a test-case failure; final verification passed in all 20. These tests may still miss behavioural faults.

Three runs switched to the stronger model, all on the same PhiCode episode. The stronger model used 34 of 3,594 model turns and 93,388 of 20,811,212 tokens. Since all switches occurred in one episode, these data do not isolate the effect of escalation.

Fig. 3 separates original smells from smells introduced during refactoring. Topological ordering fixed 9.7 introduced smells per completed run on average, compared with 10.8 for detector order and 11.2 for DAPS. BeFS averaged 15.0 across its two completed cases. Every introduced smell in these runs was later removed.

B. Ordering outcomes and overhead

The detector-order, DAPS, and topological conditions cover the same six completed episodes. DAPS used the fewest model turns (1,035) and the fewest tokens (6,263,115). Detector-order runs had the lowest elapsed time (144.8 minutes). Topological

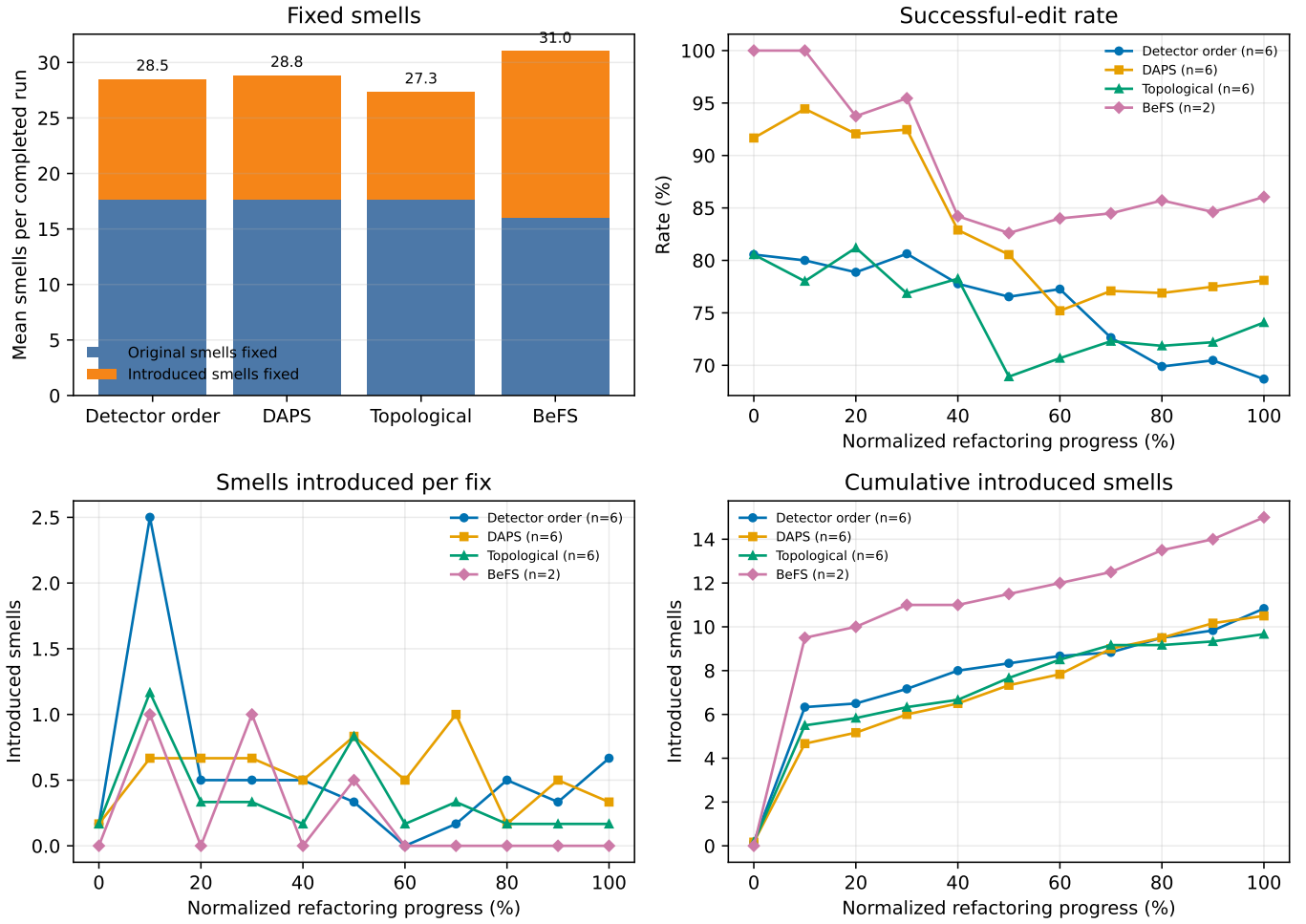


Fig. 3. Smell statistics averaged across completed cases for each planner. The fixed-smell panel reports mean counts per run. The trajectory panels normalise each run from 0% to 100% progress and average values at ten-percentage-point intervals. Detector-order runs, DAPS, and topological ordering each have six cases; BeFS has two.

order introduced the fewest smells (58, versus 65 for detector order and 67 for DAPS).

BeFS covers two episodes rather than six, so we compare the orderings on the two shared episodes. BeFS introduced 30 smells and used 39 edit tasks. DAPS introduced 23 and used 32, detector order introduced 31 and used 33, and topological ordering introduced 25 and used 42. BeFS introduced fewer smells than detector order and used fewer tasks than topological ordering. DAPS introduced fewer smells and used fewer tasks than BeFS.

The detector-order condition provides the CK baseline for comparing final states. Across the six episodes shared by detector order, DAPS, and topological execution, most final values changed little. Relative to the detector-order value, the cross-planner spread was at most 4.2% for CBO, 8.7% for WMC, 1.4% for LOC, 9.4% for class count, and 2.2% for method count. LCOM was also close in four episodes, with spreads from 1.6% to 4.2%, but differed by 19.0% and 27.7% in two episodes. Ordering had a clearer effect on intermediate outcomes than on the final CK values.

V. DISCUSSION AND FUTURE WORK

(RQ1) Planner choice. In the initial 4×10 matrix, detector order, DAPS, and topological execution each completed five cases; BeFS completed one. Across the six episodes later completed by detector order, DAPS, and topological execution, they used 92, 95, and 102 edit tasks respectively. DAPS used 1,035 model turns, compared with 1,090 for detector order and 1,103 for topological ordering. The planners differed in work and token use, but no planner led every outcome. The single run per cell and unequal completed coverage prevent a causal ranking.

(RQ2) Dependency rules. The experiment does not isolate the rules. The detector-order condition omits dependency-based planning. DAPS, topological ordering, and BeFS combine the rules with different search procedures. A clean test would run each algorithm twice, once with dependency edges and once without them.

(RQ3) Predicted and observed dependencies. The same rule graph led to different smell sequences. Across the six matched episodes, topological ordering introduced 58 smells,

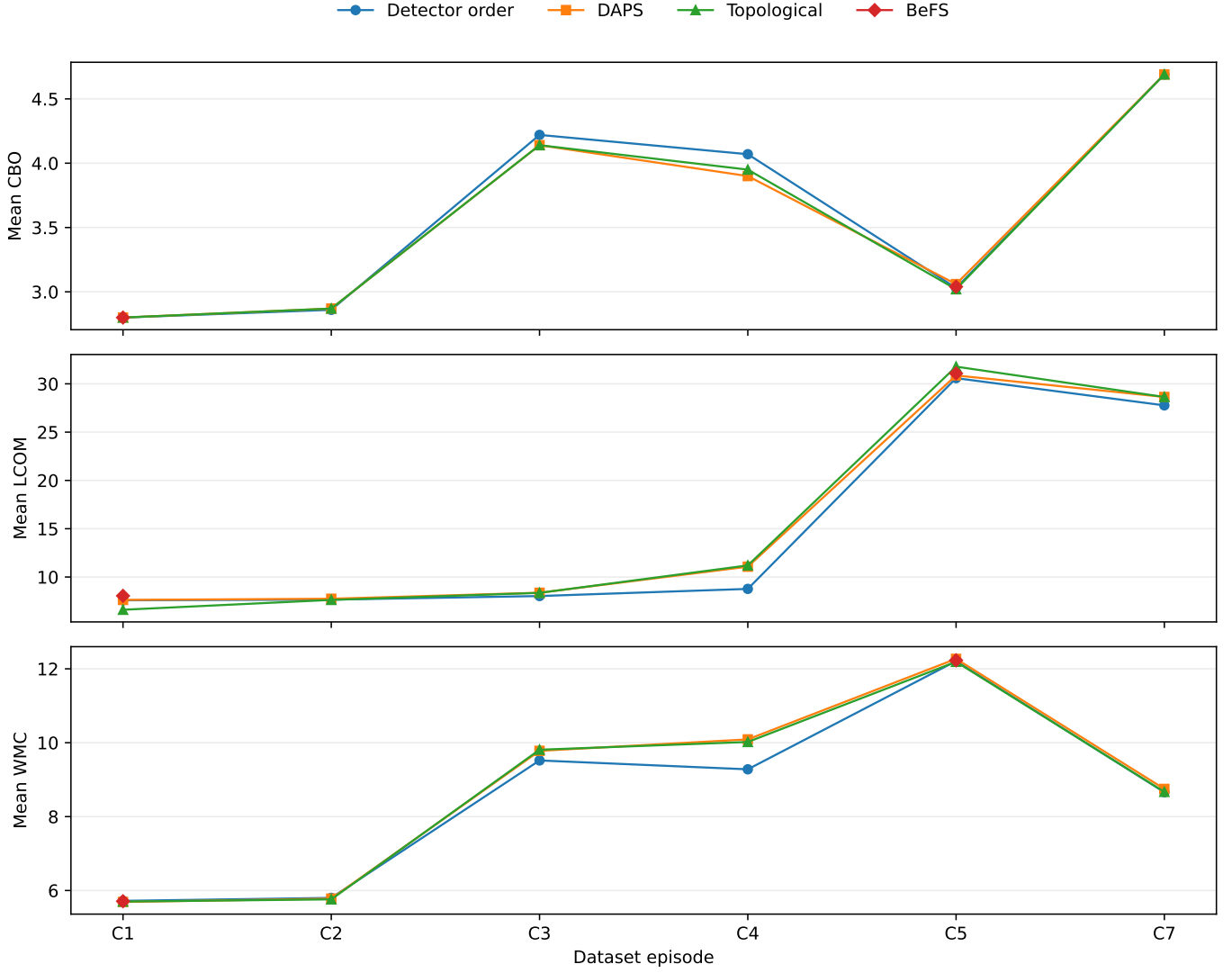


Fig. 4. Final design metrics from the experiment report. Each marker is one completed combination of planner and episode. Missing markers denote unavailable runs.

detector order 65, and DAPS 67. On the two episodes shared with BeFS, DAPS introduced 23, topological ordering 25, BeFS 30, and detector order 31. Dependency rules alone did not guarantee fewer introduced smells. The planner determined which edges became relevant, and redetection showed which predicted effects occurred. Because the rules and search procedure change together, these data cannot assign each introduced smell to one factor.

(RQ4) Structural quality. Final CBO, WMC, LOC, class count, and method count remained close across algorithms. LCOM differed more in two episodes, but no planner was consistently lower. The logs contain Organic measurements after each edit, but CK measurements only at run completion. They support a comparison of final code states, not a step-by-step CK trajectory. Measuring that trajectory requires CK collection after every accepted refactoring.

Harness recovery. Fourteen completed runs encountered

at least one compilation failure, yet all 20 completed runs removed every detected smell. The harness reported the failure, retried the same smell, and switched to a stronger model after two failures. These logs show recovery within this harness, not an advantage over another execution system.

A. Limitations and Responsible AI

The ten episodes are a selected Java-only subset of the Sousa-derived dataset. The evaluation excludes other projects and programming languages. Four episodes yielded no completed observation, and coverage differs across orderings. The dependency catalogue is incomplete, and measured edits do not calibrate its edge probabilities. Organic findings and CK metrics help identify structurally problematic code.

The task is behaviour-preserving refactoring. The harness changes production code and keeps the repository tests fixed. Passing those tests does not prove semantic equivalence or

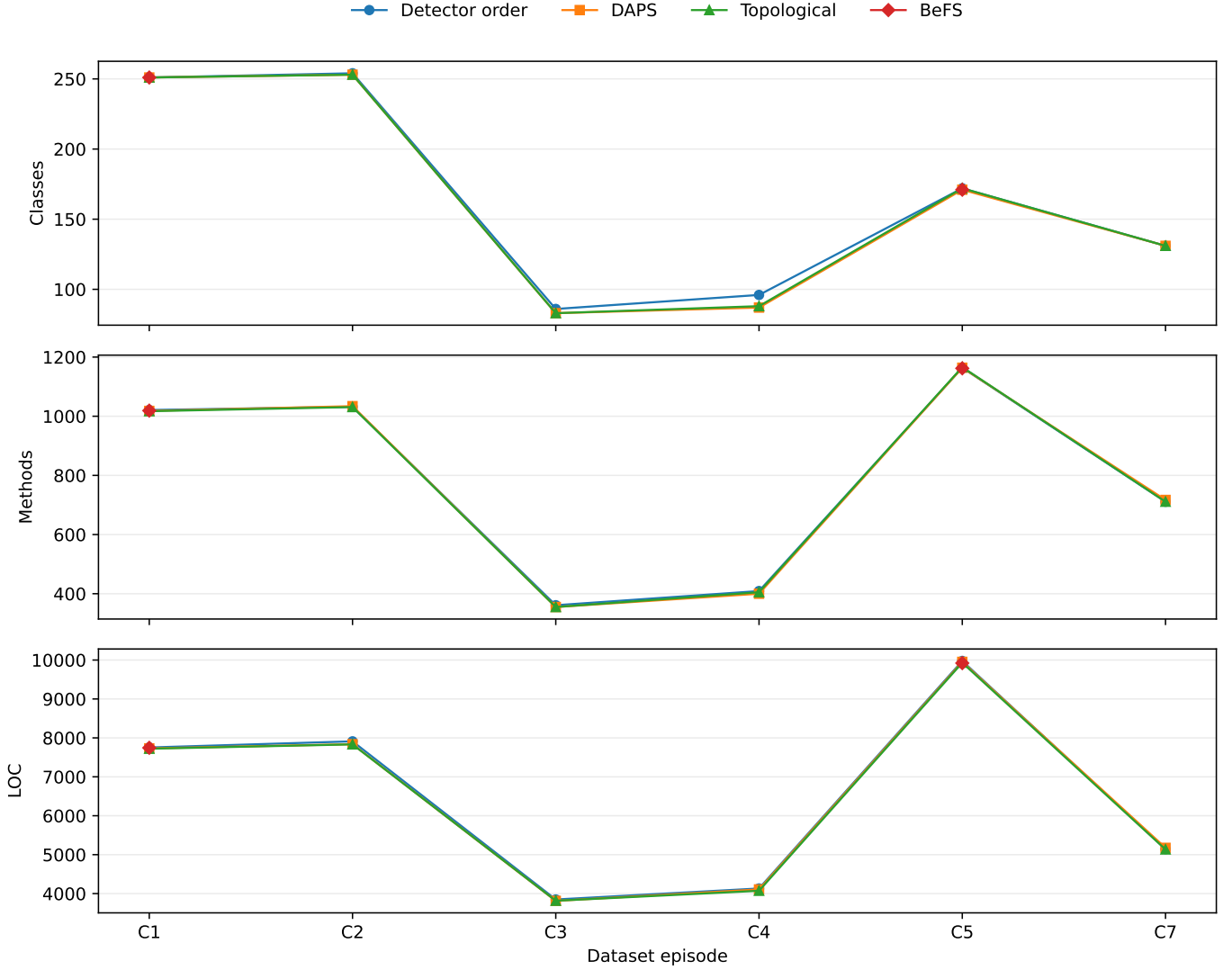


Fig. 5. Final class count, method count, and LOC from the experiment report. The values remain close across orderings within each completed episode.

adequate coverage. Future work should use change-focused coverage and mutation testing to check whether the tests execute modified code and detect injected faults. A developer must review the diff, detector output, and test result before merging a patch. The system does not deploy generated changes.

External model providers may receive source code. Users must apply their repository privacy and data-processing rules. Reproduction requires the pinned repository commits, detector and extension versions, model configuration, and execution limits. Source code, experiment configuration, and figure-generation data are available at <https://github.com/hpietukhin/refactoring-with-llms>.

Each ordering algorithm should be run with and without dependency edges. This would separate the rules from the search procedure. Frequencies measured from actual edits could then calibrate the catalogue edges.

VI. CONCLUSION

We integrated dependency-aware smell ordering with a coding-agent harness. The extension verifies each Java edit, returns the result to the model, retries failed smells, and escalates the model after two failures. Across 20 completed runs, it removed all 350 initial and 220 introduced detections despite 171 failed intermediate compilations. The harness recovered across edits but did not succeed on every edit or case. DAPS used the fewest model turns and the fewest tokens. Detector-order runs had the shortest elapsed time, while topological ordering introduced the fewest smells across their six completed episodes. On the two episodes shared with BeFS, it introduced fewer smells than detector order and used fewer edit tasks than topological ordering. Final CK metrics were mostly similar across planners. The experiment changes the rules and algorithm together, so it cannot assign their effects separately.

ACKNOWLEDGEMENT

This work was supported by the European Union NextGenerationEU via the Recovery and Resilience Plan for Slovakia through the project "InnovAlte Slovakia, Illuminating Pathways for AI-Driven Breakthroughs" (No. 09I02-03-V01-00029).

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA: Addison-Wesley, 1999.
- [2] F. Palomba, G. Bavota, M. D. Penta, F. Fasano, R. Oliveto, and A. D. Lucia, "On the diffuseness and the impact on maintainability of code smells: A large scale empirical investigation," *Empir. Softw. Eng.*, vol. 23, no. 3, pp. 1188–1221, 2018.
- [3] M. Abbes, F. Khomh, Y.-G. Guéhéneuc, and G. Antoniol, "An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension," in *Proc. CSMR*, Oldenburg, Germany, 2011, pp. 181–190.
- [4] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, "iSMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring," in *Proc. ASE*, Sacramento, CA, 2024, pp. 1345–1357.
- [5] W. Liang, J. Wang, H.-T. Zheng, Y. Li, H. Lin, H.-G. Kim, B. An, Z. Wei, and Y. Xu, "SmellDetector: Multi-label code smell detection and refactoring with large language models," in *Proc. IJCNN*, 2025, pp. 1–8.
- [6] D. Cedrim, A. Garcia, M. Mongiovi, R. Gheyi, L. Sousa, R. de Mello, B. Fonseca, M. Ribeiro, and A. Chávez, "Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects," in *Proc. ESEC/FSE*, Paderborn, Germany, 2017, pp. 465–475.
- [7] L. Markovič, "Refaktorovanie softvérových systémov pomocou XML technológií," Master's thesis, Faculty of Informatics and Information Technologies, Slovak University of Technology in Bratislava, Bratislava, Slovakia, 2016, supervisor: I. Polášek.
- [8] M. Lanza and R. Marinescu, *Object-Oriented Metrics in Practice*. Berlin: Springer, 2006.
- [9] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, 1994.
- [10] F. A. Fontana, M. V. Mäntylä, M. Zanoni, and A. Marino, "Comparing and experimenting machine learning techniques for code smell detection," *Empir. Softw. Eng.*, vol. 21, no. 3, pp. 1143–1191, 2016.
- [11] A. Yamashita and L. Moonen, "Exploring the impact of inter-smell relations on software maintainability: An empirical study," in *Proc. ICSE*, San Francisco, CA, 2013, pp. 682–691.
- [12] V. Gaba, A. Mathur, R. Singh, P. Wendell, and M. Zaharia, "Benchmarking coding agents on Databricks' multi-million line codebase," Databricks Engineering Blog, July 2026, accessed: 2026-08-27. [Online]. Available: <https://www.databricks.com/blog/benchmarking-coding-agents-databricks-multi-million-line-codebase>
- [13] Earendil Inc. and contributors, "Pi Agent Harness," Software, 2026, accessed: 2026-08-27. [Online]. Available: <https://github.com/earendil-works/pi>
- [14] Opus Research Group, "Organic: A code-smell detector for java," Software, 2022, accessed: 2026-08-27. [Online]. Available: <https://github.com/opus-research/organic-standalone>
- [15] L. Orlova and I. Polášek, "Searching for possibilities to use topological sorting in the refactoring process," in *Proceedings of the 2nd InnovAlte Conference: AI Synergy: Connecting Research, Industry, and Regional Innovation*, D. Cymbalák, Ed. Košice, Slovakia: elfa, s.r.o., Apr. 2026, pp. 108–113. [Online]. Available: <https://innovaite.sk/proceedings>
- [16] L. Sousa, D. Cedrim, A. Garcia, W. Oizumi, A. C. Bibiano, D. Oliveira, M. Kim, and A. Oliveira, "Characterizing and identifying composite refactorings: Concepts, heuristics and patterns," in *Proc. MSR*, 2020, pp. 186–197.
- [17] M. Aniche, *Java Code Metrics Calculator (CK)*, 2015, software, available at <https://github.com/mauricioaniche/ck>.